MDC/MDIO, signaling basics, 8b/10b, and basic MAC architecture

1. The big picture

A **Media Access Control (MAC)** block handles frame-level rules.

A **Physical Layer (PHY)** block turns those frame-oriented signals into real electrical signaling on the medium. A management sideband lets software read and control PHY state.

In many systems, a DMA-based data path then moves frames between memory and the MAC without burdening the CPU with every byte.

IEEE 802.3 material describes the **Media Independent Interface (MII)** family as the boundary that lets a MAC connect to different PHYs, while IEEE 802.1 material explains the **Internal Sublayer Service (ISS)** and **Enhanced Internal Sublayer Service (EISS)** as service-interface views above the raw MAC logic.

How the real Ethernet controllers add DMA engines, FIFOs, and receive/transmit queues around the MAC block, is implementation dependent at the silicon design level.

Key Questions:

1. How PHY management works through **Management Data Clock (MDC)** and **Management Data Input/Output (MDIO)**.
2. How line signaling works using schemes such as **Differential Manchester Encoding (DME)**, **Pulse Amplitude Modulation with 3 levels (PAM-3)**, and **Pulse Amplitude Modulation with 5 levels (PAM-5)**.
3. How a coding layer such as **8b/10b** prepares data for reliable serial transport.
4. How the MAC side is usually organized as service interfaces plus transmit and receive data paths.

2. MDC/MDIO from first principles

2.1 What MDC/MDIO is really for

MDC/MDIO is the classic management sideband for Ethernet PHY devices. Texas Instruments' PHY configuration note says the serial management interface is used to interrogate and control Ethernet PHYs over a shared two-wire bus, and that the interface is compatible with IEEE 802.3 Clause 22 and Clause 45. The same note explains that software uses it to configure speed, duplex, and other PHY behavior after reset.

The correct first-principles view:

MDC/MDIO is not the data path.

It is the **control and status path** between a management entity and one or more PHYs.

The Ethernet frames themselves do not flow over MDIO.

MDIO is how software reads link state, advertises capabilities, selects operating mode, accesses diagnostics, and controls PHY-specific features.

2.2 The two signals

The two wires are simple in concept. **MDC** is the clock driven by the management station. **MDIO** is the bidirectional serial data pin carrying management transactions between the station and the PHY. TI datasheets say exactly that: MDC is sourced by the external management entity, often called the station, and MDIO carries management data into and out of the PHY.

Simple intuitive description:

- MDC tells everyone **when to sample**
- MDIO carries **what to read or write**.

2.3 Clause 22 MDIO

Clause 22 is the original, simpler MDIO model.

An IEEE 802.3ca presentation summarizing the basic MDIO frame structure says the Clause 22 format includes **preamble**, **start of frame**, **operation**, **PHY address**, **register address**, **turnaround**, and **data**.

It also says the PHY address space supports up to **32 PHYs** on a single MDIO bus and the register address supports up to **32 registers**.

The spirit of Clause 22:

It is a small, simple, direct register-access model.

A management station selects one PHY address, one register address, and performs a read or write.

This was enough for classic PHY control and status, where the standard register map was relatively compact.

A useful mental model is to think of Clause 22 as the “original small register file” approach. It is clean and easy to explain, but it becomes restrictive once PHYs grow more complex and need many more control/status registers.

2.4 Clause 45 MDIO

Clause 45 extends that basic model. The same IEEE 802.3ca presentation says Clause 45 expands the operation codes, replaces the simple register-space idea with **Port Address (PRTAD)** and **Device Address (DEVAD)**, and that DEVAD corresponds to an **MDIO Manageable Device (MMD)**.

Another IEEE 802.3 management-register presentation says each Clause 45 MMD has a large address space of **0 through 65,535 registers**, each 16 bits wide, and that vendor-specific registers occupy the upper part of that space.

That is the key difference. Clause 22 says, roughly, "pick a PHY and one of its 32 standard registers." Clause 45 says, roughly, "pick a port, then pick a manageable device inside that PHY or PHY-related function, then pick one of a very large number of registers in that device."

So Clause 45 is not just "newer MDIO." It is **structured MDIO**, designed for more complex PHY architectures where different functional blocks such as **Physical Medium Attachment / Physical Medium Dependent (PMA/PMD)**, **Physical Coding Sublayer (PCS)**, Forward Error Correction, and vendor-specific logic all need substantial management space. IEEE 802.3bp draft text for 1000BASE-T1 explicitly says the Clause 45 MDIO capability provides control and status information for the PMA.

2.5 Practical coexistence: Clause 22 access to Clause 45-style space

Real devices often need to support both worlds. IEEE 802.3ah-era material on "Clause 22 access to Clause 45 registers" shows why: older CPUs and MACs often only knew how to speak Clause 22 MDIO, but newer PHYs needed the larger Clause 45 space. Modern TI automotive PHY datasheets still reflect this reality. They say registers above address `0x001F` are accessed indirectly using Clause 22 registers `0x0D` and `0x0E`, with the **DEVAD** selecting the target MMD.

An important practical point: even when a PHY is conceptually Clause 45-capable, real embedded systems sometimes reach those extended registers through a Clause 22-compatible indirect method. So one should not imagine Clause 22 and Clause 45 as two completely isolated worlds. In practice, devices often bridge between them.

3. Electrical signaling basics

3.1 Why signaling is a separate topic from framing

An Ethernet frame is a symbolic Layer 2 object. The copper pair does not see "destination address" and "payload" directly. It sees voltages changing over time. The PHY therefore needs a signaling method that can survive attenuation, noise, echo, crosstalk, and baseline wander, while also fitting the bandwidth of the medium. That is why signaling schemes such as DME, PAM-3, and PAM-5 exist. They are answers to the question: **how should binary information become a real waveform?**

3.2 DME: Differential Manchester Encoding

Here, **DME** means **Differential Manchester Encoding**. IEEE 802.3 single-pair Ethernet auto-negotiation material says BASE-T1 auto-negotiation uses **Differential Manchester encoded (DME) pages**, and describes them as **DC balanced**, **polarity insensitive**, and **simple and robust to decode**. The same material says the DME message pages carry capability information such as 100BASE-T1 ability, 1000BASE-T1 ability, master/slave configuration, and flow-control ability.

This point is easy to misunderstand, so it needs to be said clearly:

DME is not the main user-data modulation of 100BASE-T1 or 1000BASE-T1.
In the BASE-T1 context, DME is used for **auto-negotiation signaling pages**, not as the primary line modulation for payload traffic. IEEE 802.3bp auto-negotiation proposals talk specifically about receiving and decoding **DME page sequences**, and the Clause 98 tutorial notes that BASE-T1 auto-negotiation is a distinct mechanism from classic Clause 28 BASE-T negotiation.

So the most useful way to explain DME here is: it is a **robust control-signaling code** used to exchange link-partner capabilities and related information before or around normal high-speed data operation.

3.3 PAM-3: Pulse Amplitude Modulation with 3 levels

PAM-3 means each transmitted symbol can take one of **three amplitude levels** instead of only two binary levels. An IEEE 802.3 tutorial for 10BASE-T1L says that 10BASE-T1L uses **PAM-3 transmission** with **4B3T** coding.

A typical TI automotive 100BASE-T1 PHY datasheet says its 100BASE-T1 device transmits **PAM3 ternary symbols at 66.667 MHz** over an unshielded single twisted pair.

More recent IEEE 802.3dg tutorial material also states that **100BASE-T1 and 1000BASE-T1 are PAM-3**.

The right way to understand PAM-3 is not to stop at “three levels.”

The real reason to use more than two levels is that you can push more signaling information through a given bandwidth than a simple two-level scheme would allow, while still keeping the analog front end simpler than a much higher-order modulation.

In automotive single-pair Ethernet, that trade-off matters because cable bandwidth, echo cancellation complexity, and electromagnetic behavior are all tightly constrained.

The IEEE 802.3bp work on 1000BASE-T1 explicitly says PAM-3 was identified as a suitable modulation scheme.

So PAM-3 is best understood as a **multi-level amplitude signaling compromise** that fits the short-reach, echo-cancelled, single-pair Ethernet problem well.

3.4 PAM-5: Pulse Amplitude Modulation with 5 levels

PAM-5 means each symbol is drawn from **five possible amplitude levels**.

1000BASE-T uses PAM 5 coding on each pair, runs at a **125 MHz clock**, uses **four pairs**, and operates **full duplex on each pair** with heavy use of adaptive filtering to cancel echo, crosstalk, and noise. IEEE 10GBASE-T study material also notes that PAM-5 had already been used in **100BASE-T2** and **1000BASE-T**.

This gives the clean historical picture. When classic four-pair copper Ethernet had to reach 1 Gb/s over Category 5 class cabling, PAM-5 was a good fit. It offered more symbol states than binary signaling, enabling higher data throughput per baud while still being manageable with digital signal processing. But it came with the cost of significant echo and crosstalk cancellation complexity.

So PAM-5 is best understood as the **classic multilevel copper-gigabit signaling solution** for 1000BASE-T, especially on four-pair office-style twisted-pair cabling.

3.5 DME versus PAM-3 versus PAM-5

Now the distinctions become much cleaner.

- **DME** in this context is a **control / auto-negotiation coding method**, especially in BASE-T1 auto-negotiation.
- **PAM-3** is a **three-level payload signaling method** used in single-pair Ethernet families such as 10BASE-T1L, 100BASE-T1, and 1000BASE-T1.
- **PAM-5** is a **five-level payload signaling method** used historically in copper gigabit Ethernet such as 1000BASE-T.

That is the right conceptual split. DME is not “another rival modulation” to PAM-3 and PAM-5 in the same payload sense here; it is a robust signaling format for auto-negotiation pages in the BASE-T1 family.

4. 8b/10b encoding

4.1 What 8b/10b is trying to solve

8b/10b is not primarily an electrical modulation. It is a **line coding scheme**. It maps each 8-bit input byte or control character into a 10-bit transmission character. IEEE 802.3z / 10GBASE-X historical material describes 8b/10b in terms of **running disparity**, meaning the difference between the number of transmitted ones and zeros, and says the transmitter keeps the running disparity around zero while the receiver checks it. Another IEEE 802.3 text says the PCS transmission code referred to as 8B/10B is identical to that specified in Clause 36.

The deep reason for 8b/10b is that raw bit streams are inconvenient for serial link receivers.

You need enough transitions for clock recovery, and you want the signal to stay reasonably DC-balanced so the channel and receiver do not drift into trouble.

8b/10b addresses that by adding overhead and carefully choosing code groups.

8b/10b ensures good **DC balance** through the concept of running disparity.

8b/10b trades 25% line-code overhead for clock-friendly, disparity-controlled transmission characters and useful control-symbol support.

4.2 Where Ethernet uses 8b/10b

IEEE 802.3 maintenance text says the **1000BASE-X** family is adapted from Fibre Channel and uses the associated **8b/10b data coding method**, while **Clause 40 1000BASE-T does not use 8b/10b coding**.

Other IEEE material says **10GBASE-X** PHY families are also based on 8b/10b, while **10GBASE-R** uses **64b/66b** instead.

This is exactly why 8b/10b should be understood as a **general Ethernet PCS concept**, not as a universal Ethernet rule.

Some Ethernet PHY families use it, especially older or serial-fiber/backplane families like 1000BASE-X.

Others, especially copper BASE-T families such as 1000BASE-T, use very different PCS and modulation strategies.

4.3 Why to know about 8b/10b even though T1 PHYs use PAM-3

Just to contrast general data communications and automotive Ethernet:

- 8b/10b as a **general PCS line-code concept**
- PAM-3/PAM-5 as **actual electrical signaling styles on specific copper PHY families**.

A very common confusion is to mix them together and ask whether 8b/10b “is the same as” PAM-3. It is not.

8b/10b is a code mapping at the PCS layer.

PAM-3 and PAM-5 are line-level amplitude signaling schemes. They live at different conceptual points in the transmit chain.

5. Basic MAC architecture

5.1 Two different ways to discuss MAC architecture

There are really two useful viewpoints here.

The first is the **standards service view**: how the MAC appears as a service boundary to higher functions and to bridges.

The second is the **implementation data path view**: how a real controller is built with DMA engines, FIFOs, and receive/transmit queues. IEEE 802.1 documents are good for the first view, while silicon vendor application notes are clearer for the second.

5.2 ISS: Internal Sublayer Service

The **ISS**, or **Internal Sublayer Service**, is the bridge-side service view derived from the MAC Service.

An IEEE 802.1AE architectural description says the service provided to the MAC Relay Entity of a MAC Bridge is the MAC ISS specified in IEEE 802.1D, and that the ISS is derived from the MAC Service by augmenting the **MA_UNITDATA.request** and **MA_UNITDATA.indication** primitives with two parameters necessary for relay operation: **frame_type** and **frame_check_sequence**.

That means ISS is not a hardware queue or a register block.

It is a **service interface abstraction** used by bridging functions.

It says, in effect, "what extra information must be visible to internal relay functions beyond the ordinary external MAC service?"

The answer includes at least frame type and FCS awareness.

5.3 EISS: Enhanced Internal Sublayer Service

The **EISS**, or **Enhanced Internal Sublayer Service**, is the further-evolved bridge-internal service used by VLAN bridge and related media-access-independent forwarding functions.

An IEEE 802.1Q revision update says IEEE 802.1Q specifies an **Enhanced Internal Sublayer Service (EISS)** provided to the media-access-independent functions that provide frame relay in a VLAN bridge.

Extended stream-identification work for IEEE 802.1CBdb also says the extended stream-identification function can be defined using the EISS described in IEEE 802.1Q and enhanced with an extra `stream_handle` parameter.

So the clean way to present ISS versus EISS is:

- **ISS** = the internal bridge-facing version of the MAC service
- **EISS** = the richer internal service used where VLAN-aware and enhanced relay functions need more context.

Again, these are **architectural service interfaces**, not data path queues. That distinction is very important because one may confuse any "internal interface" term with a concrete hardware block.

5.4 DMA in a MAC data path

Direct Memory Access (DMA) is the mechanism that lets frame data move between system memory and the Ethernet controller without forcing the CPU to copy every word manually.

The Ethernet controller is typically composed of three main functional blocks: the **DMA**, the **MAC Transmission Layer (MTL) which sits between the** DMA and the MAC managing the internal FIFOs. It also says the embedded DMA engine has independent transmit and receive hardware. The receive path includes a **receive DMA engine**, receive FIFO, and MAC receiver, while the transmit path includes a **transmit DMA engine**, transmit FIFO, and MAC transmitter.

That gives the practical data path model:

- the MAC itself handles frame protocol behavior
- the DMA moves buffers between memory and MAC-side queues/FIFOs
- separate RX and TX data paths often exist in hardware.

So DMA is the engine that makes high-throughput Ethernet practical on embedded and processor-based systems. Without it, the CPU would waste enormous effort just shoveling frame payload bytes back and forth.

5.5 RX/TX queues and FIFOs

A real Ethernet MAC/controller usually has distinct buffering structures for receive and transmit traffic. MTL block manages the internal FIFOs for each receive and transmit path. Typical **Queue Management Unit (QMU)** manages packet traffic between MAC/PHY and the host, with built-in **TXQ** and **RXQ** packet memory.

This means RX/TX queues are the implementation-side structures where frames wait while:

- DMA fetches or writes buffers
- the MAC serializes outbound traffic
- the receiver side accumulates inbound frames before software or DMA consumes them.

In more advanced switches and controllers, there may be **multiple queues per direction** so different priorities or traffic classes can be handled separately. Multiple queues and channels may exist precisely because queue multiplicity becomes important once Ethernet controllers are used for traffic classes rather than just one undifferentiated data stream.

5.6 Putting ISS/EISS together with DMA and RX/TX queues

This is where many explanations go wrong. ISS and EISS are **architectural service abstractions** used in standards and bridge modeling. DMA and RX/TX queues are **implementation blocks** used in silicon datapaths. They are both part of “MAC architecture” in a broad sense, but they live at different abstraction levels.

A good way to explain the relationship is:

- **ISS/EISS** tell you how internal bridge or forwarding logic conceptually talks to the MAC service
- **DMA, FIFOs, RX/TX queues** tell you how actual hardware moves and buffers frame data.

6. Automotive perspective and context

6.1 Why Clause 22 and Clause 45 matter more in automotive than students first expect

Automotive Ethernet PHYs expose much more than simple link-up status. They often carry diagnostics, master/slave settings, cable-quality information, low-power state control, timestamping, and extensive vendor-specific visibility.

That is exactly why Clause 45-style structured management becomes more important in automotive PHYs. IEEE 802.3bp draft text for **1000BASE-T1** explicitly maps PMA control and status variables to Clause 45 MDIO.

Typical automotive **100BASE-T1** and **1000BASE-T1** PHY registers above the Clause 22 base range are accessed through the MMD indirect method for Clause 45-style extended register access.

So in automotive systems, MDC/MDIO is not just a lab-debug convenience. It is one of the main ways the embedded software stack configures and diagnoses the PHY layer. That includes startup mode, link partner capabilities, low-power behavior, and vendor-specific health data.

6.2 DME in automotive Ethernet

In the automotive single-pair world, DME is especially relevant because BASE-T1 auto-negotiation is **not** the same as old Clause 28 BASE-T auto-negotiation. In IEEE 802.3 single-pair Ethernet the BASE-T1 auto-negotiation is defined in **Clause 98** and is **not compatible with Clause 28**, and uses a similar base-page / next-page concept.

These pages are **Differential Manchester encoded (DME)** and are used to exchange capabilities such as 100BASE-T1 ability, 1000BASE-T1 ability, master/slave configuration, and flow control.

It is wrong to think that “DME is the modulation of the automotive data link.” It is more accurate to say:

DME is a key part of BASE-T1 link bring-up and capability exchange.

6.3 PAM-3 is central to automotive T1 PHYs

This is one of the most automotive-specific parts of your topic.

Typical Automotive 100BASE-T1 PHY transmits **PAM3 ternary symbols at 66.667 MHz** over unshielded single twisted-pair cable. IEEE 802.3 task-force material also says

100BASE-T1 and **1000BASE-T1** are PAM-3, and specific 1000BASE-T1 task-force work says PAM-3 was identified as a suitable modulation scheme for 1000BASE-T1.

So for automotive Ethernet, PAM-3 is not just another modulation example from a communications textbook. It is one of the defining electrical signaling choices behind the short-reach single-pair automotive backbone and edge links.

6.4 PAM-5 is historically important, but not the main automotive T1 story

PAM-5 matters because it explains how **1000BASE-T** over four-pair twisted pair achieved gigabit speeds using multilevel signaling and adaptive echo/crosstalk cancellation. In current automotive T1 systems, the more relevant modulation family is PAM-3 on single-pair links.

6.5 8b/10b in automotive context

8b/10b still matters as a **general Ethernet PHY / PCS concept** and for some serial interfaces and non-T1 families. It is not the line code of 100BASE-T1 or 1000BASE-T1 payload transmission. IEEE 802.3 maintenance material explicitly says the 1000BASE-X family uses the associated 8b/10b coding method, while 1000BASE-T does not.

So the most useful automotive teaching message is:

- know 8b/10b because it is part of broader Ethernet PCS history and some serial Ethernet families
- do **not** confuse it with the actual payload electrical signaling of 100BASE-T1 and 1000BASE-T1, which are PAM-3 based.

6.6 MAC architecture in automotive ECUs and switches

In automotive systems, the basic MAC architecture is often more visible to the software team than in office networking gear, because embedded engineers are dealing directly with DMA channels, RX/TX queues, timestamping, buffer ownership, and interrupt behavior. ST's Ethernet multiple-queue note and TI's EMAC guide show exactly this kind of architecture: independent TX/RX DMA engines, FIFOs, MAC logic, and queue-management structures. TI automotive PHY datasheets also show that a single automotive PHY family may support **MII**, **RMII**, **RGMII**, and **SGMII**, which means the MAC-side integration choice strongly affects platform reuse.

That is why, in automotive, "basic MAC architecture" is not purely abstract. It directly affects:

- CPU load and latency
- timestamp accuracy paths
- queue-based prioritization
- ease of moving from a 100BASE-T1 design to a 1000BASE-T1 design
- software portability across ECUs and switches.

6.7 ISS/EISS in the automotive bridge context

ISS and EISS become more relevant as soon as the system stops being a simple endpoint and becomes a bridge, switch, or time-sensitive forwarding element.

The IEEE 802.1Q update says EISS is the service provided to the media-access-independent functions that perform frame relay in a VLAN bridge.

FRER stream-identification work then shows EISS being extended with extra metadata such as `stream_handle`.

That is directly relevant to automotive switches because modern vehicles increasingly rely on VLAN-aware switching, Time-Sensitive Networking, and stream-aware forwarding. So while ISS/EISS may sound abstract, they are part of the conceptual foundation for how standards describe what an automotive Ethernet switch is doing internally.

7. Summary

MDC/MDIO is the PHY management sideband. **Clause 22** gives the original small register model with fields like PHY address and register address.

Clause 45 expands that model with **Port Address (PRTAD)** and **MDIO Manageable Device (MMD / DEVAD)** addressing and very large register spaces, which is why it is so useful for complex PHYs.

DME here means **Differential Manchester Encoding** for BASE-T1 auto-negotiation pages.

PAM-3 is the main multilevel signaling family for 10BASE-T1L, 100BASE-T1, and 1000BASE-T1.

PAM-5 is the classic multilevel signaling used in 1000BASE-T over four pairs. **8b/10b** is a PCS line code with running-disparity control, widely used in families like 1000BASE-X, but not the payload line code of 1000BASE-T or T1 PHYs.

For **basic MAC architecture**, the most useful split is between the **standards-side service view** and the **implementation datapath view**. **ISS** and **EISS** are internal MAC/bridge service abstractions used by IEEE 802.1 bridging architecture.

DMA, **RX/TX FIFOs**, and **RX/TX queues** are the concrete data path blocks used by real controllers and switches to move and buffer frames efficiently.

In automotive systems, all of this matters even more because PHY management, one-pair signaling, queue behavior, and timestamp paths are part of the actual ECU and switch design, not just textbook diagrams.